

Webprogramozás

Féléves feladat

2026.tavaszi

Készítette: **Koncz József**

Szak: Programtervező Informatikus

Neptunkód: **KUETUO**

Sárospatak, 2026

Tartalom

A feladat leírása	3
1. Az alkalmazás fő funkciói.....	3
2. Az alkalmazás felépítése	3
3. Adatmodell és adatszolgáltatás	4
a) Az adatbázis szerkezete:.....	4
4. Szerszámok listázása	5
5. Készlet mennyiségének módosítása	5
6. Új szerszám rögzítése.....	6
7. Telepítés és futtatás	7
8. Összefoglalás.....	7

A feladat leírása

A Kéziszerszám Nyilvántartó alkalmazás egy Angular keretrendszerben készült webes alkalmazás, amely lehetővé teszi kéziszerszámok nyilvántartását, kezelését és készletkövetését. Az alkalmazás JSON Server segítségével biztosítja az adatok perzisztens tárolását. A fejlesztés során Angular standalone komponenseket, TypeScriptet, valamint HTTP kommunikációra a HttpClient szolgáltatást használtuk.

1. Az alkalmazás fő funkciói

Az alkalmazás három fő funkciót valósít meg. Először is megjeleníti a nyilvántartott szerszámok listáját táblázatos formában. Másodszor lehetővé teszi a készleten lévő mennyiség módosítását + és - gombokkal. Harmadszor pedig új szerszámok rögzítését biztosítja űrlap segítségével, amely tartalmazza a megfelelő validációkat és az egyedi név ellenőrzését is.

Kéziszerszám nyilvántartó

[Szerszámok listája](#)[Új szerszám rögzítése](#)

Szerszámok listája

ID	Név	Típus	Ár (Ft)	Készleten	Műveletek
1	Kalapács	ütő	2500	12	 
2	Csavarhúzó	rögzítéstechnika	1200	5	 
ZwQ8a5L_bik	Kombináltfogó	fogó	5825	2	 
zxgrPLB5A7A	Csípőfogó	vágó	4700	2	 
69TDcKjO2Q4	Sniccer	vágó	1240	2	 

2. Az alkalmazás felépítése

Az alkalmazás két fő komponensből épül fel. A ListaSzerszamComponent felelős a szerszámok listájának megjelenítéséért és a készlet módosításáért, míg a NewSzerszamComponent az új szerszámok rögzítését végzi. A routing konfiguráció az app.config.ts fájlban található, ahol a gyökér útvonal a lista komponensre, a /new útvonal pedig az új szerszám komponensre mutat:

```
const routes: Routes = [
  { path: '', component: ListaSzerszamComponent, pathMatch: 'full' },
  { path: 'new', component: NewSzerszamComponent },
  { path: '**', redirectTo: '' }
];
```

3. Adatmodell és adatszolgáltatás

A szerszámok adatmodelljét a Szerszam interfész határozza meg, amely tartalmazza az egyedi azonosítót, a nevet, a típust, az árat és a készleten lévő darabszámot. Az adatszolgáltatást a SzerszamService végzi, amely a HTTP kommunikációt kezeli a JSON Server REST API-jával.

A szolgáltatás négy fő metódust biztosít. A getSzerszamok() lekéri az összes szerszámot, az addSzerszam() újat hoz létre, az updateSzerszam() frissíti a készlet mennyiségét, az existsByName() pedig ellenőrzi, hogy egy adott névvel már létezik-e szerszám a nyilvántartásban:

```
private apiUrl = 'http://localhost:3000/szerszamok';
```

```
getSzerszamok(): Observable<Szerszam[]> {
  return this.http.get<Szerszam[]>(this.apiUrl);
}
```

```
updateSzerszam(id:      number,      changes:      Partial<Szerszam>):
Observable<Szerszam> {
  return this.http.patch<Szerszam>(`${this.apiUrl}/${id}`, changes);
}
```

a) Az adatbázis szerkezete:

```
{
  "szerszamok": [
    {
      "id": "1",
```

```

        "nev": "Kalapács",
        "tipus": "ütő",
        "ar": 2500,
        "készleten": 12
    }
],
"$schema": "./node_modules/json-server/schema.json"
}

```

4. Szerszámok listázása

A lista komponens a BehaviorSubject segítségével biztosítja a reaktív adatfrissítést. A komponens inicializálásakor betölti a szerszámokat, majd az async pipe segítségével jeleníti meg az adatokat a sablonban. A trackByFn függvény segít az Angularnek optimalizálni a listázást, mivel egyedi azonosító alapján követi nyomon az elemeket:

```

private szerszamokSubject = new BehaviorSubject<Szerszam[]>([]);
szerszamok$: Observable<Szerszam[]> =
this.szerszamokSubject.asObservable();

```

```

loadSzerszamok(): void {
    this.szerszamService.getSzerszamok().subscribe({
        next: (data) => this.szerszamokSubject.next(data)
    });
}

```

5. Készlet mennyiségének módosítása

A készlet módosítása a táblázat minden sorában található + és - gombok segítségével történik. Amikor a felhasználó rákattint valamelyik gombra, a komponens meghívja a SzerszamService updateSzerszam() metódusát, amely egy PATCH kérést küld a szerver felé. A művelet sikere után a program újratölti a teljes listát, így a UI mindig a legfrissebb adatokat mutatja:

```

increaseQuantity(sz: Szerszam): void {
    const updatedCount = sz.készleten + 1;

```

```

    this.szerszamService.updateSzerszam(sz.id!, { készleten:
updatedCount }).subscribe({
    next: () => this.loadSzerszamok(),
    error: (err) => alert('Hiba történt a készlet növelése közben!')
});
}

```

A csökkentés gomb letiltásra kerül, ha a készlet értéke nulla, így a felhasználó nem léptethet negatív értékre.

ZwQ8a5L_bik	Kombináltfogó	fogó	5825	0	- +
-------------	---------------	------	------	---	---

6. Új szerszám rögzítése

Az új szerszám rögzítéséhez egy űrlap áll rendelkezésre, amely minden adatot bekér a szerszámmal kapcsolatban. Az űrlap három szintű ellenőrzést végez. Először a `validateForm()` metódus ellenőrzi, hogy minden mező ki van-e töltve, és a megfelelő formátumú-e. Ezután a szolgáltatás `existsByName()` metódusa megvizsgálja, hogy a megadott név már nem szerepel-e a nyilvántartásban. Végül, ha minden rendben van, a `addSzerszam()` metódus elmenti az új szerszámot:

Új szerszám rögzítése

Név:	Kombináltfogó
Típus:	fogó
Ár (Ft):	5825
Készleten (db):	2
Hozzáadás	

7. Telepítés és futtatás

Az alkalmazás futtatásához két terminálablakra van szükség. Az elsőben el kell indítani a JSON Server szolgáltatást a **npm run server** paranccsal, amely a 3000-es porton biztosítja a REST API-t. A második terminálban az Angular alkalmazás indítható az **npm run start** paranccsal, amely alapértelmezés szerint a 4200-as porton teszi elérhetővé az alkalmazást. Ezt követően a böngészőben a <http://localhost:4200> címen érhető el a Kéziszerszám Nyilvántartó alkalmazás.

8. Összefoglalás

A Kéziszerszám Nyilvántartó alkalmazás sikeresen megvalósította az összes kitűzött funkcionalitást. A szerszámok táblázatos listázása, a készletmennyiség valós idejű módosítása, valamint az új szerszámok validációval ellátott rögzítése is működik. Az Angular standalone komponenseken alapuló felépítés moduláris és könnyen továbbfejleszthető. A JSON Server használata egyszerű, SQL szerver nélküli adatkezelést tesz lehetővé, ami kiválóan alkalmas kis- és közepes méretű alkalmazások prototípusainak készítéséhez.

Koncz József

Programtervező Informatikus, BSc, hallgató